Practical – 6

Rendering External JSON Data using Fetch API, Search and Filter

Problem Definition

Display event lists, student profiles, and FAQs from external JSON files in the StudentHub portal using the Fetch API. Implement dynamic rendering, search, filter, sorting, and pagination, show loading and error states, and keep the JavaScript modular. Use events.json, students.json, and faqs.json with at least 15 records each as test data.

A. Theory

1. How is JSON fetched, parsed, and rendered?

Answer: fetch(url) sends an HTTP request and returns a Promise of a Response. The code checks response.ok (a 404 or 500 does not throw by itself), then calls response.json(), which reads the body and parses it into JavaScript arrays and objects (it throws if the JSON is invalid). The array is stored in a state object. Each record is turned into a DOM element by a render function (renderEvent, renderStudent, renderFaq) that creates elements with createElement and textContent and inserts them into the results container with replaceChildren().

2. Which array methods are used for search, filter, sort, and pagination?

Answer: Search uses filter() with every() and includes() on a lower-case text built with map() and join(). Filtering uses filter() with strict comparison. Sorting uses a copy made with the spread operator and sort() with localeCompare() for text or subtraction for numbers. Pagination uses slice(startIndex, startIndex + pageSize) and Math.ceil(total / pageSize). Dropdown options are made with map(), Set, and sort(). Rendering uses map() to create elements.

3. How are loading and error states handled?

Answer: Before the request starts, a status message with a spinner is shown and the result list is marked aria-busy="true". The request is inside try/catch. If the network fails, the status is not OK, or the JSON is invalid, an error box with role="alert", a clear message, and a Try again button is shown. If an earlier response was cached in localStorage, that data is shown with a notice instead. When no record matches the search, a friendly empty-state message is displayed.

4. How is modularity maintained in JavaScript files?

Answer: The code is split into ES modules with import and export: api.js (fetching and caching), utils.js (pure array functions for search, filter, sort, and pagination), views.js (DOM rendering), and main.js (state, events, and connecting everything). Each file has one responsibility, utils.js has no DOM code and can be tested in the console or Node, and the page loads only main.js with <script type="module">.

5. What is JSON and how is it different from a JavaScript object?

Answer: JSON (JavaScript Object Notation) is a text format for data. It requires double-quoted keys and strings and does not allow comments, functions, undefined, or trailing commas. JSON.parse() converts JSON text to an object and JSON.stringify() converts an object to JSON text.

6. What is a Promise and what do async and await do?

Answer: A Promise represents a value that will be available later. async marks a function that returns a Promise, and await pauses that function until a Promise settles, which lets asynchronous code read like normal top-to-bottom code. Errors are handled with try/catch.

7. Why does fetch() fail on a file:// page?

Answer: Browsers block fetch() for local files for security reasons. The site must be opened through an HTTP server such as VS Code Live Server, python -m http.server, or XAMPP. The error box in this practical mentions this when the page is opened from file://.

8. Why are textContent and createElement used instead of innerHTML?

Answer: Data from a JSON file is not trusted. innerHTML would interpret tags inside the data and could run harmful scripts (XSS). textContent always treats the data as plain text.

9. What is debouncing?

Answer: Debouncing delays a function until the user has stopped doing something for a short time. The search box waits 250 ms after the last keystroke before filtering, which avoids unnecessary re-rendering on every key press.

10. How is the page number calculated?

Answer: totalPages = Math.ceil(total / pageSize) and the first record of a page is (page − 1) × pageSize. For example, 17 events with 6 per page give 3 pages, and page 3 shows records 13 to 17.

B. Implementation / Output / Result

Extend the StudentHub project. Add a data folder with three JSON files, split the JavaScript into modules, add explorer.html, and append the new styles to css/style.css. Serve the folder with a local web server (for example the VS Code Live Server extension) because fetch() does not work from file:// addresses.

1. Project folder structure

```
StudentHub/
├── index.html
├── events.html
├── explorer.html      (new: search, filter, sort, pagination)
├── ... (other pages)
├── css/
│   └── style.css      (previous styles + Practical 6 additions)
├── data/
│   ├── events.json      (17 records)
│   ├── students.json    (16 records)
│   └── faqs.json        (17 records)
└── js/
    ├── script.js        (Practical 4: theme, menu)
    ├── main.js          (new: state, events, loading and error states)
    └── modules/
        ├── api.js       (fetch + localStorage cache)
        ├── utils.js     (search, filter, sort, paginate)
```

```
└── views.js     (DOM rendering)
```

2. JSON data files

Each file is an array of records. Records are shown one per line here to save space; the same data may be formatted in the usual indented style.

data/events.json


```
[
  {"id":1,"title":"Web Development Workshop","category":"Workshop","date":"2026-11-
12","time":"10:00","venue":"Lab 3","seats":60,"description":"Hands-on session on HTML, CSS and
JavaScript. Bring your laptop."},
  {"id":2,"title":"24-Hour Hackathon","category":"Technical","date":"2026-11-
20","time":"09:00","venue":"Main Auditorium","seats":120,"description":"Teams of up to four
build a solution to a real campus problem."},
  {"id":3,"title":"Career Talk: Software Engineering","category":"Career","date":"2026-11-
28","time":"15:00","venue":"Seminar Hall","seats":200,"description":"Alumni share interview
tips and career paths in software engineering."},
  {"id":4,"title":"Annual Cultural Night","category":"Cultural","date":"2026-12-
05","time":"18:00","venue":"Open Air Theatre","seats":500,"description":"Music, dance and drama
performances by student clubs."},
  {"id":5,"title":"Inter-College Cricket Tournament","category":"Sports","date":"2026-12-
08","time":"08:30","venue":"College Ground","seats":150,"description":"Knockout tournament with
eight participating colleges."},
  {"id":6,"title":"Python for Data Analysis","category":"Workshop","date":"2026-11-
15","time":"14:00","venue":"Lab 5","seats":45,"description":"Learn pandas and basic charts
using real datasets."},
  {"id":7,"title":"Placement Drive: Day 1","category":"Career","date":"2026-12-
10","time":"09:30","venue":"Training and Placement Cell","seats":300,"description":"Aptitude
test and group discussion rounds for final-year students."},
  {"id":8,"title":"Robotics Expo","category":"Technical","date":"2026-12-
14","time":"11:00","venue":"Mechanical Block","seats":180,"description":"Student-built robots
on display with live line-follower races."},
  {"id":9,"title":"Photography Contest","category":"Cultural","date":"2026-11-
18","time":"10:00","venue":"Art Gallery","seats":80,"description":"Theme: Campus Life. Submit
up to three photographs."},
  {"id":10,"title":"Cyber Security Awareness Session","category":"Technical","date":"2026-11-
22","time":"12:00","venue":"Seminar Hall","seats":220,"description":"Safe browsing, phishing
detection and password hygiene."},
  {"id":11,"title":"Yoga and Wellness Camp","category":"Sports","date":"2026-11-
25","time":"07:00","venue":"Sports Complex","seats":100,"description":"Morning yoga and stress-
management tips before examinations."},
  {"id":12,"title":"Resume Building Clinic","category":"Career","date":"2026-11-
30","time":"13:00","venue":"Room 204","seats":50,"description":"Mentors review resumes and
LinkedIn profiles one-to-one."},
  {"id":13,"title":"UI/UX Design Sprint","category":"Workshop","date":"2026-12-
02","time":"10:30","venue":"Design Studio","seats":40,"description":"Design and prototype a
mobile app screen in one day."},
  {"id":14,"title":"Open Mic and Poetry Evening","category":"Cultural","date":"2026-12-
12","time":"17:30","venue":"Library Lawn","seats":90,"description":"Share poems, stories or
stand-up in a relaxed setting."},
  {"id":15,"title":"Cloud Computing Bootcamp","category":"Technical","date":"2026-12-
16","time":"09:00","venue":"Lab 2","seats":70,"description":"Introduction to cloud services
with free-tier practical exercises."},
  {"id":16,"title":"Basketball Friendly Match","category":"Sports","date":"2026-12-
18","time":"16:00","venue":"Indoor Court","seats":60,"description":"Friendly match between
faculty and student teams."},
  {"id":17,"title":"Entrepreneurship Meetup","category":"Career","date":"2026-12-
20","time":"15:30","venue":"Incubation Centre","seats":110,"description":"Founders talk about
starting up while still in college."}
]
```


data/students.json


```
[
  {"id":1,"name":"Riya Patel","department":"Computer
Engineering","year":2,"cgpa":9.1,"email":"riya.patel@example.com","country":"India","state":"Gu
jarat","city":"Surat","skills":["HTML","CSS","JavaScript"]},
```

```
  {"id":2,"name":"Aarav Shah","department":"Information
Technology","year":3,"cgpa":8.4,"email":"aarav.shah@example.com","country":"India","state":"Guj
arat","city":"Ahmedabad","skills":["Python","SQL"]},
  {"id":3,"name":"Meera Desai","department":"Computer
Engineering","year":4,"cgpa":8.9,"email":"meera.desai@example.com","country":"India","state":"G
ujarat","city":"Surat","skills":["Java","React"]},
  {"id":4,"name":"Kabir Mehta","department":"Mechanical
Engineering","year":2,"cgpa":7.6,"email":"kabir.mehta@example.com","country":"India","state":"G
ujarat","city":"Vadodara","skills":["CAD","MATLAB"]},
  {"id":5,"name":"Ananya Joshi","department":"Electronics and
Communication","year":3,"cgpa":8.2,"email":"ananya.joshi@example.com","country":"India","state"
:"Gujarat","city":"Rajkot","skills":["Embedded C","IoT"]},
  {"id":6,"name":"Rohan Verma","department":"Civil
Engineering","year":1,"cgpa":7.9,"email":"rohan.verma@example.com","country":"India","state":"M
aharashtra","city":"Mumbai","skills":["AutoCAD","Surveying"]},
  {"id":7,"name":"Isha Nair","department":"Information
Technology","year":2,"cgpa":9.3,"email":"isha.nair@example.com","country":"India","state":"Maha
rashtra","city":"Pune","skills":["Node.js","MongoDB"]},
  {"id":8,"name":"Dev Kapoor","department":"Computer
Engineering","year":1,"cgpa":8,"email":"dev.kapoor@example.com","country":"India","state":"Guja
rat","city":"Surat","skills":["C++","Git"]},
  {"id":9,"name":"Sana Sheikh","department":"Electronics and
Communication","year":4,"cgpa":8.7,"email":"sana.sheikh@example.com","country":"India","state":
"Gujarat","city":"Ahmedabad","skills":["VLSI","Arduino"]},
  {"id":10,"name":"Neel Trivedi","department":"Mechanical
Engineering","year":3,"cgpa":7.4,"email":"neel.trivedi@example.com","country":"India","state":"
Gujarat","city":"Vadodara","skills":["SolidWorks","Thermodynamics"]},
  {"id":11,"name":"Priya Iyer","department":"Information
Technology","year":4,"cgpa":9,"email":"priya.iyer@example.com","country":"India","state":"Mahar
ashtra","city":"Mumbai","skills":["Cloud","Docker"]},
  {"id":12,"name":"Yash Rathod","department":"Civil
Engineering","year":2,"cgpa":7.8,"email":"yash.rathod@example.com","country":"India","state":"G
ujarat","city":"Surat","skills":["Estimation","Staad Pro"]},
  {"id":13,"name":"Emily Carter","department":"Computer
Engineering","year":3,"cgpa":8.5,"email":"emily.carter@example.com","country":"USA","state":"Ca
lifornia","city":"San Jose","skills":["Python","Machine Learning"]},
  {"id":14,"name":"Daniel Brooks","department":"Information
Technology","year":2,"cgpa":8.1,"email":"daniel.brooks@example.com","country":"USA","state":"Te
xas","city":"Austin","skills":["JavaScript","UX"]},
  {"id":15,"name":"Olivia Reed","department":"Computer
Engineering","year":4,"cgpa":9.2,"email":"olivia.reed@example.com","country":"UK","state":"Engl
and","city":"London","skills":["Java","Cyber Security"]},
  {"id":16,"name":"Harry Wilson","department":"Electronics and
Communication","year":1,"cgpa":7.7,"email":"harry.wilson@example.com","country":"UK","state":"E
ngland","city":"Manchester","skills":["Circuits","Python"]}
]
```


data/faqs.json


```
[
  {"id":1,"topic":"Registration","question":"How do I register on StudentHub?","answer":"Open
the Register page, fill in your details, accept the terms and submit the form."},
  {"id":2,"topic":"Registration","question":"Can I change my email address after
registering?","answer":"Yes. Contact the portal administrator through the Contact page with
your new email address."},
  {"id":3,"topic":"Registration","question":"Why is my password rejected?","answer":"Passwords
need at least 8 characters with uppercase, lowercase, a digit and a special character."},
  {"id":4,"topic":"Account","question":"How do I reset a forgotten password?","answer":"Use the
Forgot password link on the login page and follow the instructions sent to your email."},
  {"id":5,"topic":"Account","question":"Can I use StudentHub on my phone?","answer":"Yes. The
portal is responsive and works on modern mobile browsers."},
  {"id":6,"topic":"Account","question":"How do I switch to the dark theme?","answer":"Use the
theme button in the header. Your choice is remembered on the same device."},
  {"id":7,"topic":"Events","question":"How can I join a campus event?","answer":"Open the
Events page, choose an event and follow its registration link."},
  {"id":8,"topic":"Events","question":"Is there a fee for workshops?","answer":"Most workshops
are free for registered students. Fees, if any, are shown on the event card."},
  {"id":9,"topic":"Events","question":"Can I cancel my event registration?","answer":"Yes.
Write to the event coordinator at least one day before the event."},
  {"id":10,"topic":"Results","question":"Where can I see my results?","answer":"Results are
listed on the Dashboard page after login."},
```

```
  {"id":11,"topic":"Results","question":"How do I request a re-evaluation?","answer":"Submit
the re-evaluation form at the examination section within seven days of the result."},
  {"id":12,"topic":"Results","question":"How is CGPA calculated?","answer":"CGPA is the credit-
weighted average of grade points across all completed semesters."},
  {"id":13,"topic":"Library","question":"What are the library timings?","answer":"The library
is open from 8 AM to 6 PM, and until 10 PM during examinations."},
  {"id":14,"topic":"Library","question":"How many books can I borrow?","answer":"Undergraduate
students can borrow up to three books for fourteen days."},
  {"id":15,"topic":"Library","question":"How do I renew a book?","answer":"Renew from the
library counter or through the library desk before the due date."},
  {"id":16,"topic":"Placement","question":"When does the placement drive start?","answer":"The
main placement drive begins in December. Watch the notice banner for dates."},
  {"id":17,"topic":"Placement","question":"Who can attend placement drives?","answer":"Final-
year students with no active backlogs can attend."}
]
```

3. Data Explorer page (save as explorer.html)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>StudentHub | Data Explorer</title>
  <link rel="stylesheet" href="css/style.css">
  <script src="js/script.js" defer></script>
  <script type="module" src="js/main.js"></script>
</head>
<body>
  <a class="skip-link" href="#main-content">Skip to main content</a>

  <header class="site-header">
    <div class="container header-row">
      <p class="brand"><strong>StudentHub</strong></p>
      <div class="header-controls">
        <button class="icon-btn" id="theme-toggle" type="button" aria-label="Dark theme" aria-
pressed="false">Dark</button>
        <button class="icon-btn menu-toggle" id="menu-toggle" type="button" aria-label="Toggle
navigation menu" aria-expanded="false" aria-controls="main-nav">&#9776;</button>
      </div>
      <nav class="main-nav" id="main-nav" aria-label="Main navigation">
        <ul class="nav-list">
          <li><a href="index.html">Home</a></li>
          <li><a href="register.html">Register</a></li>
          <li><a href="events.html">Events</a></li>
          <li><a href="explorer.html" aria-current="page">Explorer</a></li>
          <li><a href="contact.html">Contact</a></li>
        </ul>
      </nav>
    </div>
  </header>

  <main id="main-content" class="container">
    <h1>Data Explorer</h1>
    <p class="muted">Events, student profiles and FAQs are loaded from external JSON files
using the Fetch API.</p>

    <!-- Dataset switcher -->
    <div class="tabs" role="group" aria-label="Choose a dataset">
      <button class="button secondary" type="button" data-dataset="events" aria-
pressed="true">Events</button>
      <button class="button secondary" type="button" data-dataset="students" aria-
pressed="false">Students</button>
      <button class="button secondary" type="button" data-dataset="faqs" aria-
pressed="false">FAQs</button>
    </div>

    <!-- Search, filter, sort controls -->
    <form class="toolbar panel" id="toolbar" role="search" aria-label="Search and filter">
```

```
      <div class="tool">
        <label for="search">Search</label>
        <input type="search" id="search" placeholder="Type to search..." autocomplete="off">
      </div>
      <div class="tool">
        <label for="filter" id="filter-label">Category</label>
        <select id="filter"></select>
      </div>
      <div class="tool">
        <label for="sort">Sort by</label>
        <select id="sort"></select>
      </div>
      <div class="tool">
        <label for="page-size">Per page</label>
        <select id="page-size">
          <option value="6" selected>6</option>
          <option value="9">9</option>
          <option value="12">12</option>
        </select>
      </div>

      <!-- Dependent dropdowns (shown for students only) -->
      <div class="location-filters" id="location-filters" hidden>
        <div class="tool">
          <label for="country">Country</label>
          <select id="country"></select>
        </div>
        <div class="tool">
          <label for="state">State</label>
          <select id="state" disabled></select>
        </div>
        <div class="tool">
          <label for="city">City</label>
          <select id="city" disabled></select>
        </div>
      </div>

      <div class="tool tool-action">
        <button class="button secondary" type="button" id="clear-filters">Clear
filters</button>
      </div>
    </form>

    <p id="cache-notice" class="alert notice" role="status" hidden></p>
    <p id="count" class="count" role="status" aria-live="polite"></p>

    <!-- Loading and error states -->
    <p id="status" class="loading" role="status">Loading...</p>
    <div id="error-box" class="alert error" role="alert" hidden>
      <p id="error-text"></p>
      <button class="button" type="button" id="retry">Try again</button>
    </div>

    <!-- Rendered results -->
    <div id="results" class="card-grid" aria-busy="true"></div>
    <nav id="pagination" class="pagination" aria-label="Pagination"></nav>
  </main>

  <footer class="site-footer">
    <div class="container">
      <p>&copy; 2026 StudentHub | Dynamic rendering with the Fetch API</p>
    </div>
  </footer>
</body>
</html>
```

4. Additional CSS (append to css/style.css)

```
/* ===== Practical 6 additions: append to css/style.css ===== */
/* (colour tokens --danger, --danger-bg, --success, --field-border and the .alert,
.button.small
   and form control styles come from Practical 5) */

.tabs { display: flex; flex-wrap: wrap; gap: .5rem; margin: 1rem 0; }
.tabs .button[aria-pressed="true"] { background: var(--primary); color: #fff; border-color:
var(--primary); }

/* Toolbar: search, filter, sort */
.toolbar { display: grid; grid-template-columns: 1fr; gap: .75rem 1rem; margin-bottom: 1rem; }
.tool label { display: block; font-weight: bold; margin-bottom: .25rem; }
.tool input, .tool select { width: 100%; padding: .55rem .7rem; font: inherit; color: var(--
text); background: var(--surface); border: 2px solid var(--field-border); border-radius: .4rem;
}
.tool select:disabled { opacity: .6; cursor: not-allowed; }
.tool-action { align-self: end; }
.location-filters { display: grid; grid-template-columns: 1fr; gap: .75rem 1rem; grid-column: 1
/ -1; }
.location-filters[hidden] { display: none; }

/* Status messages */
.count { margin: .5rem 0 1rem; font-weight: bold; min-height: 1.5em; }
.loading { display: flex; align-items: center; gap: .6rem; }
.loading::before { content: ""; width: 1.1rem; height: 1.1rem; border: 3px solid var(--border);
border-top-color: var(--primary); border-radius: 50%; animation: spin .8s linear infinite; }
@keyframes spin { to { transform: rotate(360deg); } }
.alert.notice { border: 2px solid var(--field-border); background: var(--surface); color: var(-
-text); }
.empty { grid-column: 1 / -1; padding: 1.5rem; text-align: center; background: var(--surface);
border: 1px dashed var(--border); border-radius: .75rem; }

/* Cards and chips */
.badge { display: inline-block; padding: .1rem .6rem; border-radius: 1rem; font-size: .8rem;
font-weight: bold; background: var(--primary); color: #fff; }
.card h3 { margin: .5rem 0 .25rem; }
.card .meta { margin-bottom: 0; font-size: .9rem; }
.chips { display: flex; flex-wrap: wrap; gap: .4rem; list-style: none; margin: .5rem 0;
padding: 0; }
.chips li { border: 1px solid var(--border); border-radius: .3rem; padding: .1rem .5rem; font-
size: .85rem; }

/* FAQ list (native details/summary) */
.faq-list { display: grid; gap: .75rem; }
.faq-details { background: var(--surface); border: 1px solid var(--border); border-radius:
.5rem; padding: .75rem 1rem; }
.faq-details summary { cursor: pointer; font-weight: bold; }
.faq-details p { margin: .5rem 0 0; }

/* Pagination */
.pagination { display: flex; flex-wrap: wrap; justify-content: center; align-items: center;
gap: .4rem; margin: 1.5rem 0; }
.page-btn { min-width: 2.5rem; padding: .45rem .75rem; font: inherit; color: var(--text);
background: var(--surface); border: 1px solid var(--field-border); border-radius: .4rem;
cursor: pointer; }
.page-btn:hover:not(:disabled) { border-color: var(--primary); }
.page-btn.active { background: var(--primary); color: #fff; border-color: var(--primary); font-
weight: bold; }
.page-btn:disabled { opacity: .5; cursor: not-allowed; }
.page-gap { padding: 0 .25rem; }

@media (min-width: 700px) {
  .toolbar { grid-template-columns: 2fr 1.5fr 1.5fr 1fr; }
  .location-filters { grid-template-columns: repeat(3, 1fr); }
  .tool-action { grid-column: auto; }
}
```

5. Fetch and cache module (save as js/modules/api.js)

```
/* api.js: fetching JSON with error handling and a localStorage cache (advanced extension) */

const CACHE_PREFIX = 'studenthub-cache:';

function readCache(url) {
  try {
    const raw = localStorage.getItem(CACHE_PREFIX + url);
    return raw ? JSON.parse(raw) : null;          // { savedAt, data }
  } catch (error) {
    return null;                                  // storage blocked or corrupted
  }
}

function saveCache(url, data) {
  try {
    localStorage.setItem(CACHE_PREFIX + url, JSON.stringify({ savedAt: Date.now(), data }));
  } catch (error) {
    /* ignore: quota exceeded or storage blocked; the page still works */
  }
}

export function clearCache() {
  try {
    Object.keys(localStorage)
      .filter((key) => key.startsWith(CACHE_PREFIX))
      .forEach((key) => localStorage.removeItem(key));
  } catch (error) {
    /* ignore */
  }
}

/* Returns { data, source: 'network' | 'cache', savedAt }.
   Throws only when the network fails AND nothing is cached. */
export async function fetchJSON(url) {
  try {
    const response = await fetch(url, { cache: 'no-cache' });
    if (!response.ok) {
      throw new Error('Server answered with HTTP ' + response.status + ' for ' + url);
    }
    const data = await response.json();               // throws SyntaxError for invalid JSON
    if (!Array.isArray(data)) {
      throw new Error('Unexpected format in ' + url + ': an array of records was expected.');
    }
    saveCache(url, data);
    return { data, source: 'network', savedAt: Date.now() };
  } catch (error) {
    const cached = readCache(url);
    if (cached && Array.isArray(cached.data)) {
      return { data: cached.data, source: 'cache', savedAt: cached.savedAt };
    }
    throw error;
  }
}
```

6. Array helper module (save as js/modules/utils.js)

```
/* utils.js: pure array helpers for search, filter, sort and pagination (no DOM, easy to test)
*/

/* SEARCH: every word typed must appear in at least one of the chosen fields. */
export function searchItems(items, query, fields) {
  const words = query.toLowerCase().split(/\s+/).filter(Boolean);
  if (words.length === 0) return items;
  return items.filter((item) => {
    const haystack = fields
```

```
      .map((field) => (Array.isArray(item[field]) ? item[field].join(' ') : String(item[field]
?? '')))
      .join(' ')
      .toLowerCase();
    return words.every((word) => haystack.includes(word));
  });
}

/* FILTER: keep items whose field equals the chosen value ('' or 'all' means no filter). */
export function filterBy(items, field, value) {
  if (!value || value === 'all') return items;
  return items.filter((item) => String(item[field]) === String(value));
}

/* SORT: returns a new array so the original data is never changed. */
export function sortItems(items, key, direction = 'asc', type = 'text') {
  const factor = direction === 'desc' ? -1 : 1;
  return [...items].sort((a, b) => {
    if (type === 'number') return (a[key] - b[key]) * factor;
    return String(a[key]).localeCompare(String(b[key]), 'en', { sensitivity: 'base', numeric:
true }) * factor;
  });
}

/* PAGINATE: slice() one page out of the array. */
export function paginate(items, page, pageSize) {
  const total = items.length;
  const totalPages = Math.max(1, Math.ceil(total / pageSize));
  const current = Math.min(Math.max(1, page), totalPages);
  const startIndex = (current - 1) * pageSize;
  return {
    items: items.slice(startIndex, startIndex + pageSize),
    page: current,
    totalPages,
    total,
    from: total === 0 ? 0 : startIndex + 1,
    to: Math.min(startIndex + pageSize, total)
  };
}

/* Page numbers to show, with '...' for gaps: e.g. [1, '...', 4, 5, 6, '...', 12] */
export function pageWindow(current, totalPages) {
  if (totalPages <= 7) return Array.from({ length: totalPages }, (_, i) => i + 1);
  const pages = new Set([1, totalPages, current - 1, current, current + 1]);
  const sorted = [...pages].filter((p) => p >= 1 && p <= totalPages).sort((a, b) => a - b);
  const result = [];
  sorted.forEach((p, i) => {
    if (i > 0 && p - sorted[i - 1] > 1) result.push('...');
    result.push(p);
  });
  return result;
}

/* DROPDOWN OPTIONS: unique, sorted values of one field (uses map + Set). */
export function uniqueValues(items, field) {
  return [...new Set(items.map((item) => item[field]))].sort((a, b) =>
String(a).localeCompare(String(b)));
}

/* Wait until the user stops typing before running a function. */
export function debounce(fn, delay = 250) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delay);
  };
}
```

```
export function formatDate(iso) {
  const date = new Date(iso + 'T00:00:00');
  return date.toLocaleDateString('en-GB', { day: 'numeric', month: 'short', year: 'numeric' });
}
```

7. Rendering module (save as js/modules/views.js)

```
/* views.js: turns data into DOM. Elements are created with textContent, never innerHTML,
   so text from a JSON file cannot inject HTML or scripts. */
import { formatDate, pageWindow } from './utils.js';

/* Small helper: el('p', { class: 'muted', text: 'Hello' }, childNode) */
export function el(tag, props = {}, ...children) {
  const node = document.createElement(tag);
  Object.entries(props).forEach(([key, value]) => {
    if (key === 'class') node.className = value;
    else if (key === 'text') node.textContent = value;
    else node.setAttribute(key, value);
  });
  children.forEach((child) => node.append(child));
  return node;
}

export function renderEvent(event) {
  return el('article', { class: 'card' },
    el('span', { class: 'badge', text: event.category }),
    el('h3', { text: event.title }),
    el('p', { class: 'muted' },
      el('time', { datetime: event.date, text: formatDate(event.date) }),
      document.createTextNode(' at ' + event.time + ' | ' + event.venue)),
    el('p', { text: event.description }),
    el('p', { class: 'meta', text: 'Seats: ' + event.seats })
  );
}

export function renderStudent(student) {
  const skills = el('ul', { class: 'chips', 'aria-label': 'Skills' },
    ...student.skills.map((skill) => el('li', { text: skill })));
  return el('article', { class: 'card' },
    el('span', { class: 'badge', text: student.department }),
    el('h3', { text: student.name }),
    el('p', { class: 'muted', text: 'Year ' + student.year + ' | CGPA ' +
student.cgpa.toFixed(1) }),
    el('p', { text: student.city + ', ' + student.state + ', ' + student.country }),
    skills,
    el('p', { class: 'meta' }, el('a', { href: 'mailto:' + student.email, text: student.email
}))
  );
}

export function renderFaq(faq) {
  return el('details', { class: 'faq-details' },
    el('summary', { text: faq.question }),
    el('p', { class: 'muted', text: 'Topic: ' + faq.topic }),
    el('p', { text: faq.answer })
  );
}

/* Replace the contents of a container with the rendered items */
export function renderList(container, items, renderItem) {
  container.replaceChildren(...items.map(renderItem));
}

/* Pagination buttons: Previous, page numbers, Next */
export function renderPagination(nav, info) {
  nav.replaceChildren();
  if (info.total === 0 || info.totalPages <= 1) return;
```

```
  const button = (label, page, options = {}) => {
    const btn = el('button', { class: 'page-btn', type: 'button', text: label, 'data-page':
String(page) });
    if (options.label) btn.setAttribute('aria-label', options.label);
    if (options.current) { btn.setAttribute('aria-current', 'page');
btn.classList.add('active'); }
    if (options.disabled) btn.disabled = true;
    return btn;
  };

  nav.append(button('Previous', info.page - 1, { disabled: info.page === 1, label: 'Previous
page' }));
  pageWindow(info.page, info.totalPages).forEach((p) => {
    if (p === '...') nav.append(el('span', { class: 'page-gap', text: '...', 'aria-hidden':
'true' }));
    else nav.append(button(String(p), p, { current: p === info.page, label: 'Page ' + p }));
  });
  nav.append(button('Next', info.page + 1, { disabled: info.page === info.totalPages, label:
'Next page' }));
}

/* Fill a <select> with options (first option = "All") */
export function fillSelect(select, values, allLabel, selected = '') {
  select.replaceChildren(el('option', { value: '', text: allLabel }),
    ...values.map((v) => el('option', { value: String(v), text: String(v) })));
  select.value = selected;
}
```

8. Main script (save as js/main.js)

```
/* main.js: connects the modules to the page (state, events, loading and error handling) */
import { fetchJSON } from './modules/api.js';
import { searchItems, filterBy, sortItems, paginate, uniqueValues, debounce } from
'./modules/utils.js';
import { renderEvent, renderStudent, renderFaq, renderList, renderPagination, fillSelect } from
'./modules/views.js';

const $ = (selector) => document.querySelector(selector);

/* One configuration object per dataset keeps the rest of the code generic */
const DATASETS = {
  events: {
    url: 'data/events.json', noun: 'events', render: renderEvent,
    searchFields: ['title', 'category', 'venue', 'description'],
    filterField: 'category', filterLabel: 'All categories', location: false,
    sorts: [
      { value: 'date-asc', label: 'Date: earliest first', key: 'date', dir: 'asc', type: 'text'
},
      { value: 'date-desc', label: 'Date: latest first', key: 'date', dir: 'desc', type: 'text'
},
      { value: 'title-asc', label: 'Title: A to Z', key: 'title', dir: 'asc', type: 'text' },
      { value: 'seats-desc', label: 'Seats: most first', key: 'seats', dir: 'desc', type:
'number' }
    ]
  },
  students: {
    url: 'data/students.json', noun: 'students', render: renderStudent,
    searchFields: ['name', 'department', 'city', 'state', 'country', 'skills', 'email'],
    filterField: 'department', filterLabel: 'All departments', location: true,
    sorts: [
      { value: 'name-asc', label: 'Name: A to Z', key: 'name', dir: 'asc', type: 'text' },
      { value: 'name-desc', label: 'Name: Z to A', key: 'name', dir: 'desc', type: 'text' },
      { value: 'cgpa-desc', label: 'CGPA: high to low', key: 'cgpa', dir: 'desc', type:
'number' },
      { value: 'year-asc', label: 'Year: lowest first', key: 'year', dir: 'asc', type: 'number'
}
    ]
  },
  faqs: {
```

```
    url: 'data/faqs.json', noun: 'FAQs', render: renderFaq,
    searchFields: ['question', 'answer', 'topic'],
    filterField: 'topic', filterLabel: 'All topics', location: false,
    sorts: [
      { value: 'id-asc', label: 'Default order', key: 'id', dir: 'asc', type: 'number' },
      { value: 'question-asc', label: 'Question: A to Z', key: 'question', dir: 'asc', type:
'text' },
      { value: 'topic-asc', label: 'Topic: A to Z', key: 'topic', dir: 'asc', type: 'text' }
    ]
  }
};

const state = {
  dataset: 'events', all: [],
  q: '', filter: '', country: '', stateName: '', city: '',
  sort: '', page: 1, pageSize: 6
};

const ui = {};

function cacheElements() {
  Object.assign(ui, {
    tabs: document.querySelectorAll('[data-dataset]'),
    search: $('#search'), filter: $('#filter'), filterLabel: $('#filter-label'),
    sort: $('#sort'), pageSize: $('#page-size'), clear: $('#clear-filters'),
    locationBox: $('#location-filters'),
    country: $('#country'), stateSelect: $('#state'), city: $('#city'),
    results: $('#results'), pagination: $('#pagination'),
    status: $('#status'), count: $('#count'),
    errorBox: $('#error-box'), errorText: $('#error-text'), retry: $('#retry'),
    notice: $('#cache-notice')
  });
}

/* ---------- Loading, error and empty states ---------- */
function showLoading() {
  ui.errorBox.hidden = true;
  ui.notice.hidden = true;
  ui.results.setAttribute('aria-busy', 'true');
  ui.results.replaceChildren();
  ui.pagination.replaceChildren();
  ui.status.hidden = false;
  ui.status.textContent = 'Loading ' + DATASETS[state.dataset].noun + '...';
  ui.count.textContent = '';
}

function showError(error) {
  ui.status.hidden = true;
  ui.results.setAttribute('aria-busy', 'false');
  let message = 'Could not load the data. ' + error.message;
  if (location.protocol === 'file:') {
    message += ' (Open the site through a local web server such as VS Code Live Server; fetch()
is blocked on file:// pages.)';
  }
  ui.errorText.textContent = message;
  ui.errorBox.hidden = false;
  ui.retry.focus();
}

/* ---------- Load a dataset ---------- */
async function loadDataset(name) {
  state.dataset = name;
  Object.assign(state, { q: '', filter: '', country: '', stateName: '', city: '', page: 1 });
  const cfg = DATASETS[name];

  ui.tabs.forEach((tab) => tab.setAttribute('aria-pressed', String(tab.dataset.dataset ===
name)));
  ui.search.value = '';
```

```
  ui.filterLabel.textContent = cfg.filterField.charAt(0).toUpperCase() +
cfg.filterField.slice(1);
  ui.locationBox.hidden = !cfg.location;
  ui.results.className = name === 'faqs' ? 'faq-list' : 'card-grid';

  // sort dropdown for this dataset
  ui.sort.replaceChildren(...cfg.sorts.map((s) => new Option(s.label, s.value)));
  state.sort = cfg.sorts[0].value;

  showLoading();
  try {
    const result = await fetchJSON(cfg.url);
    state.all = result.data;
    fillSelect(ui.filter, uniqueValues(state.all, cfg.filterField), cfg.filterLabel);
    if (cfg.location) refreshLocationOptions();
    if (result.source === 'cache') {
      ui.notice.textContent = 'Showing saved data from ' + new
Date(result.savedAt).toLocaleString() +
        ' because the server could not be reached.';
      ui.notice.hidden = false;
    }
    ui.status.hidden = true;
    ui.results.setAttribute('aria-busy', 'false');
    update();
  } catch (error) {
    showError(error);
  }
}

/* ---------- Dependent dropdowns: country -> state -> city ---------- */
function refreshLocationOptions() {
  const inCountry = filterBy(state.all, 'country', state.country);
  const inState = filterBy(inCountry, 'state', state.stateName);

  fillSelect(ui.country, uniqueValues(state.all, 'country'), 'All countries', state.country);

  fillSelect(ui.stateSelect, state.country ? uniqueValues(inCountry, 'state') : [], 'All
states', state.stateName);
  ui.stateSelect.disabled = !state.country;

  fillSelect(ui.city, state.stateName ? uniqueValues(inState, 'city') : [], 'All cities',
state.city);
  ui.city.disabled = !state.stateName;
}

/* ---------- Search -> filter -> sort -> paginate -> render ---------- */
function update() {
  const cfg = DATASETS[state.dataset];
  const sortRule = cfg.sorts.find((s) => s.value === state.sort) || cfg.sorts[0];

  let items = searchItems(state.all, state.q, cfg.searchFields);
  items = filterBy(items, cfg.filterField, state.filter);
  if (cfg.location) {
    items = filterBy(items, 'country', state.country);
    items = filterBy(items, 'state', state.stateName);
    items = filterBy(items, 'city', state.city);
  }
  items = sortItems(items, sortRule.key, sortRule.dir, sortRule.type);

  const info = paginate(items, state.page, state.pageSize);
  state.page = info.page;

  if (info.total === 0) {
    ui.results.replaceChildren();
    const empty = document.createElement('p');
    empty.className = 'empty';
```

```
    empty.textContent = 'No ' + cfg.noun + ' match your search. Try different words or clear
the filters.';
    ui.results.append(empty);
    ui.count.textContent = '0 ' + cfg.noun + ' found';
  } else {
    renderList(ui.results, info.items, cfg.render);
    ui.count.textContent = 'Showing ' + info.from + ' to ' + info.to + ' of ' + info.total + '
' + cfg.noun;
  }
  renderPagination(ui.pagination, info);
}

/* ---------- Events ---------- */
function bindEvents() {
  $('#toolbar').addEventListener('submit', (event) => event.preventDefault());   // Enter must
not reload the page

  ui.tabs.forEach((tab) => tab.addEventListener('click', () =>
loadDataset(tab.dataset.dataset)));

  ui.search.addEventListener('input', debounce(() => { state.q = ui.search.value.trim();
state.page = 1; update(); }, 250));
  ui.filter.addEventListener('change', () => { state.filter = ui.filter.value; state.page = 1;
update(); });
  ui.sort.addEventListener('change', () => { state.sort = ui.sort.value; state.page = 1;
update(); });
  ui.pageSize.addEventListener('change', () => { state.pageSize = Number(ui.pageSize.value);
state.page = 1; update(); });

  ui.country.addEventListener('change', () => {
    state.country = ui.country.value; state.stateName = ''; state.city = ''; state.page = 1;
    refreshLocationOptions(); update();
  });
  ui.stateSelect.addEventListener('change', () => {
    state.stateName = ui.stateSelect.value; state.city = ''; state.page = 1;
    refreshLocationOptions(); update();
  });
  ui.city.addEventListener('change', () => { state.city = ui.city.value; state.page = 1;
update(); });

  ui.clear.addEventListener('click', () => {
    Object.assign(state, { q: '', filter: '', country: '', stateName: '', city: '', page: 1 });
    ui.search.value = ''; ui.filter.value = '';
    if (DATASETS[state.dataset].location) refreshLocationOptions();
    update();
  });

  // Event delegation: one listener for every pagination button
  ui.pagination.addEventListener('click', (event) => {
    const btn = event.target.closest('.page-btn');
    if (!btn || btn.disabled) return;
    state.page = Number(btn.dataset.page);
    update();
    $('#count').scrollIntoView({ behavior: 'smooth', block: 'nearest' });
  });

  ui.retry.addEventListener('click', () => loadDataset(state.dataset));
}

document.addEventListener('DOMContentLoaded', () => {
  cacheElements();
  bindEvents();
  state.pageSize = Number(ui.pageSize.value);
  loadDataset('events');
});
```

10. Extensions

Intermediate: Dependent dropdowns for country, state, and city are provided in the Students view. State is enabled only after a country is chosen and lists only the states of that country; City is enabled only after a state is chosen. Changing a parent resets its children. The options are derived from the data itself with filter(), map(), and Set.

Advanced: Every successful response is saved in localStorage (key studenthub-cache:<url>) together with the time. If a later request fails, fetchJSON() returns the cached copy and the page shows a notice with the time the data was saved, giving an offline-like display. All reads and writes are inside try/catch so that blocked storage never breaks the page.

11. Test cases and results

The expected results below were calculated by running the helper functions on the actual JSON files, so they should match the counts shown on the page.

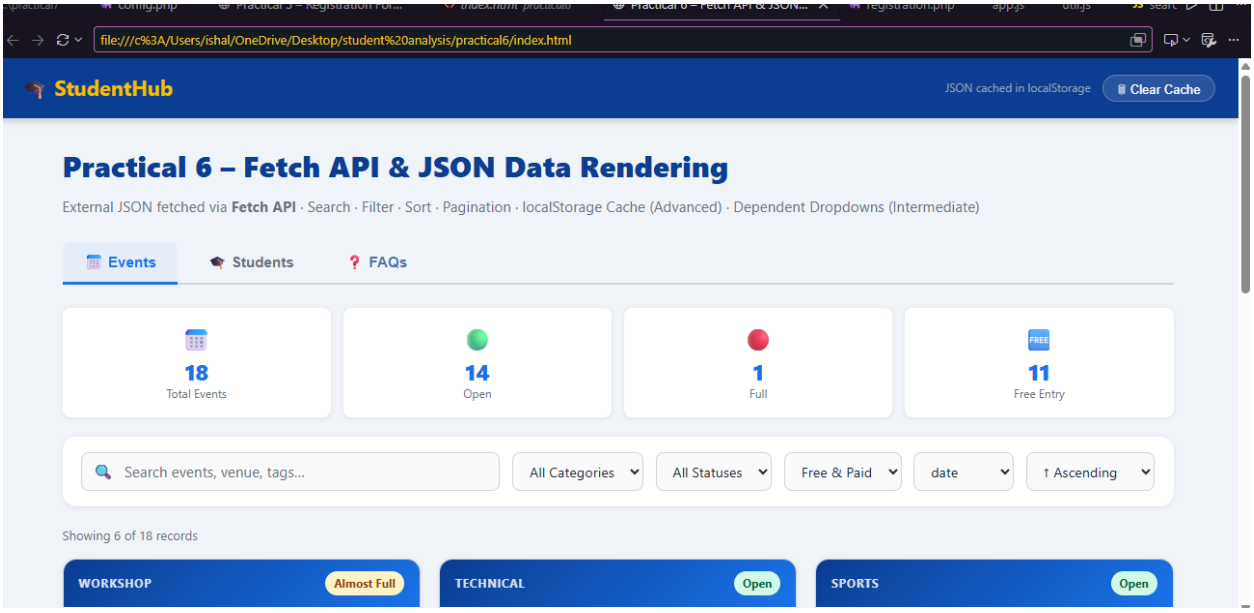
12. Browser console testing

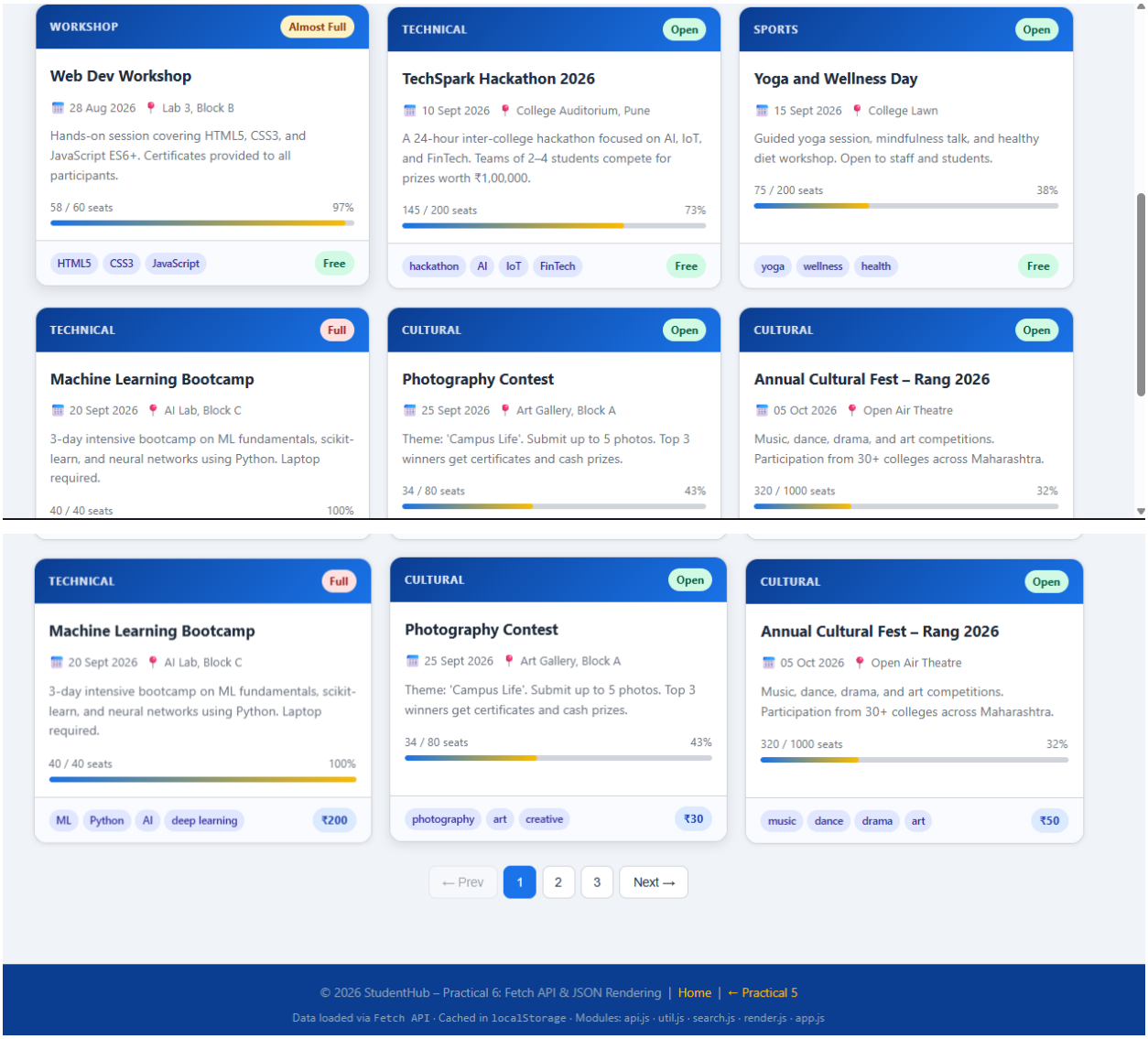
Open Developer Tools (F12). In the Network tab confirm that the three JSON files are loaded with status 200, then try these checks in the Console:

```
const events = await (await fetch('data/events.json')).json();
events.length                                          // 17
events.filter(e => e.category === 'Career').length     // 4
[...events].sort((a, b) => b.seats - a.seats)[0].title // event with the most seats
Math.ceil(events.length / 6)                           // 3 pages
Object.keys(localStorage).filter(k => k.startsWith('studenthub-cache:'))   // cached URLs
// Test the error state: Network tab -> Offline, then press "Try again"
```

C. Evidence of Work (Screenshots)

Replace each frame below with your own screenshot. Also include the three JSON files in VS Code and the Network tab showing the JSON responses.





E. Challenges Faced

1. fetch() does not reject on HTTP errors such as 404, so response.ok had to be checked manually.

2. Combining search, filter, sort, and pagination in the correct order, and resetting to page 1 when any control changes.

3. Keeping dependent dropdowns consistent when a parent value is changed or cleared.

4. Avoiding XSS by not using innerHTML with data from JSON.

5. Opening the page from file:// blocks fetch(), so a local server was needed.

F. Learning Outcomes

1. Fetched, parsed, and rendered external JSON data with the Fetch API and async/await.

2. Used map, filter, sort, slice, and Set to implement search, filter, sorting, pagination, and dropdowns.

3. Handled loading, error, and empty states in a user-friendly way.

4. Organised JavaScript into reusable ES modules.

5. Cached data in localStorage for an offline-like experience.

G. Viva Questions

What does response.json() return?

Answer: A Promise that resolves to the parsed JavaScript value.

Why check response.ok?

Answer: fetch() only rejects on network failure; HTTP errors like 404 still resolve, so they must be checked.

What is the difference between map() and forEach()?

Answer: map() returns a new array of results; forEach() returns nothing.

Why is [...items].sort() used instead of items.sort()?

Answer: sort() changes the original array; copying first keeps the loaded data unchanged.

How does slice() help in pagination?

Answer: slice(start, end) returns the part of the array for one page without changing the array.

What is the formula for the total number of pages?

Answer: Math.ceil(totalRecords / pageSize).

Why use a Set in uniqueValues()?

Answer: A Set stores each value once, which removes duplicates for the dropdown options.

What is the benefit of ES modules?

Answer: Separate files with their own scope, clear imports and exports, and easier reuse and testing.

Why is script type="module" deferred by default?

Answer: Module scripts run after the HTML is parsed, so the DOM is ready.

What is event delegation in this practical?

Answer: One click listener on the pagination nav handles every page button.

Why is the search input debounced?

Answer: It waits for the user to stop typing to avoid unnecessary re-rendering.

What happens if the JSON file has a syntax error?

Answer: response.json() throws, the catch block runs, and the error box (or cached data) is shown.

H. Result

Events, student profiles, and FAQs were loaded from external JSON files with the Fetch API and rendered dynamically. Search, filter, sorting, pagination, dependent dropdowns, loading and error states, and a localStorage cache were implemented with modular ES6 JavaScript. The test table and screenshots demonstrate the final result.

Faculty Remarks

(Keep this section blank for faculty.)